prefill completions. The probability obtained by repeated sampling is closer to what we directly aim to measure, but is naively expensive to compute. For most of our experiments we will rely on the behaviors that can be computed with logprobs to efficiently validate our forecasting methodology, but we extend to general correctness in Section 4.5.

Evaluation. Our primary evaluation metric is the accuracy of our forecasts. To capture the accuracy of the forecast, we measure the both *average absolute error*: the average absolute difference between the predicted and actual worst-query risks, and the *average absolute log error*, the average absolute difference between the log of the predicted and log of actual worst-query risks (in base 10). We report both errors since they capture failures in different regimes; log error captures difference in small probabilities, while standard absolute error captures differences in large probabilities.

Log-normal baseline. We compare our forecasts to a simpler parametric forecasting baseline that directly models distribution of negative log elicitation probabilities as log normal, or equivalently the distribution of elicitation scores as normally distributed. Specifically, we fit a normal distribution to the m observed scores ψ_i in our training set by computing the sample mean μ and standard deviation σ. This distribution ensures that the underlying elicitation probabilities are always valid. Under this assumption, we can analytically compute the expected maximum over n samples from this distribution, and compute aggregate risk by repeatedly sampling from this distribution. The log-normal method helps us assess the impact of forecasting the extreme quantiles, rather than extrapolating from average behavior.

4.2. Forecasting worst-query-risk

We first test whether we can predict the worst-query risk: the maximum elicitation probability over n deployment queries, using only m evaluation queries. Intuitively, this is a proxy for the "most effective jailbreak" at deployment.

Since the true worst-query risk is a random variable, we simulate multiple independent evaluation sets and deployment sets by partitioning the all generated queries into as many non-overlapping $(m + n)$ sets as possible, and make forecasts for each individually.

Settings. We measure across all combinations of evaluation size $m \in \{100, 500, 1000\}$, deployment size $n \in \{10000, 20000, \ldots, 90000\}$, and models (Claude 3.5 Sonnet (Anthropic, 2024b), Claude 3 Haiku (Anthropic, 2024a), and their two corresponding base models.

Results. We find that our forecasts are high-quality across all settings (Figure 3a). The average absolute log error is 1.7 for the Gumbel-tail method, compared to 2.4 for the log-normal method.[3] We also find that the Gumbel-tail forecasts tend to improve disproportionately as we increase the evaluation size, and are within an order of magnitude of the actual worst-query risk 72% of the time. See Appendix C.2.1 for more results.

We also study *how* different methods make errors; underestimates in particular pose safety risks, since they give developers a false sense of security. We find that the Gumbel-tail method tends to underestimate the actual probability only 34% of the time, compared to 72% for the log-normal, and the log-normal tends to produce larger-magnitude underestimates than the Gumbel-tail method. However, this suggests there is room to improve both methods as they are biased (an unbiased method should underestimate 50% of the time).

While it is impossible to make perfect forecasts for this task—the maximum elicitation probability over n deployment queries is a random variable—our results suggest we can nonetheless make high-quality forecasts.

4.3. Forecasting behavior frequency

We next forecast the behavior frequency: the fraction of queries with elicitation probability over some threshold τ. This forecasts the probability that each deployment query routinely exhibits the behavior.

We would like to evaluate our forecasts in settings where all elicitation probabilities on the evaluation set are below some threshold, but some elicitation at deployment crosses a relatively large threshold. Since the full output probabilities tend to be small, we focus on the probability of high-information keywords, which tend to be larger.

Settings. We measure across 1000 randomly sampled evaluation sets for each of the 19 substances, thresholds $\tau \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, and number of evaluation queries $m \in \{100, 200, 500, 1000\}$. We make forecasts whenever the fraction of queries for which the keyword probabilities exceed τ is less than $1/m$.

Results. We find that we can effectively predict behavior frequencies for behaviors that do not appear during evaluation across all settings (Figure 3b). The Gumbel-tail method has average absolute log errors on individual forecasts ranging from 0.84 to 0.76 as m ranges from 100 to 1000, compared to 3.31 to 4.04 for the log-normal method.[4] The average forecast—the average (in log space) over all random evaluation sets for the same settings—leads to a factor-of-two improvement for the Gumbel-tail method, while only slightly decreasing the error of the log-normal method. See Appendix C.2.2 for more results.

[3] The average absolute errors are all less than 0.02

[4] The average absolute error in this setting is uniformly small, since the ground truth and forecasts are less than $1/m$.

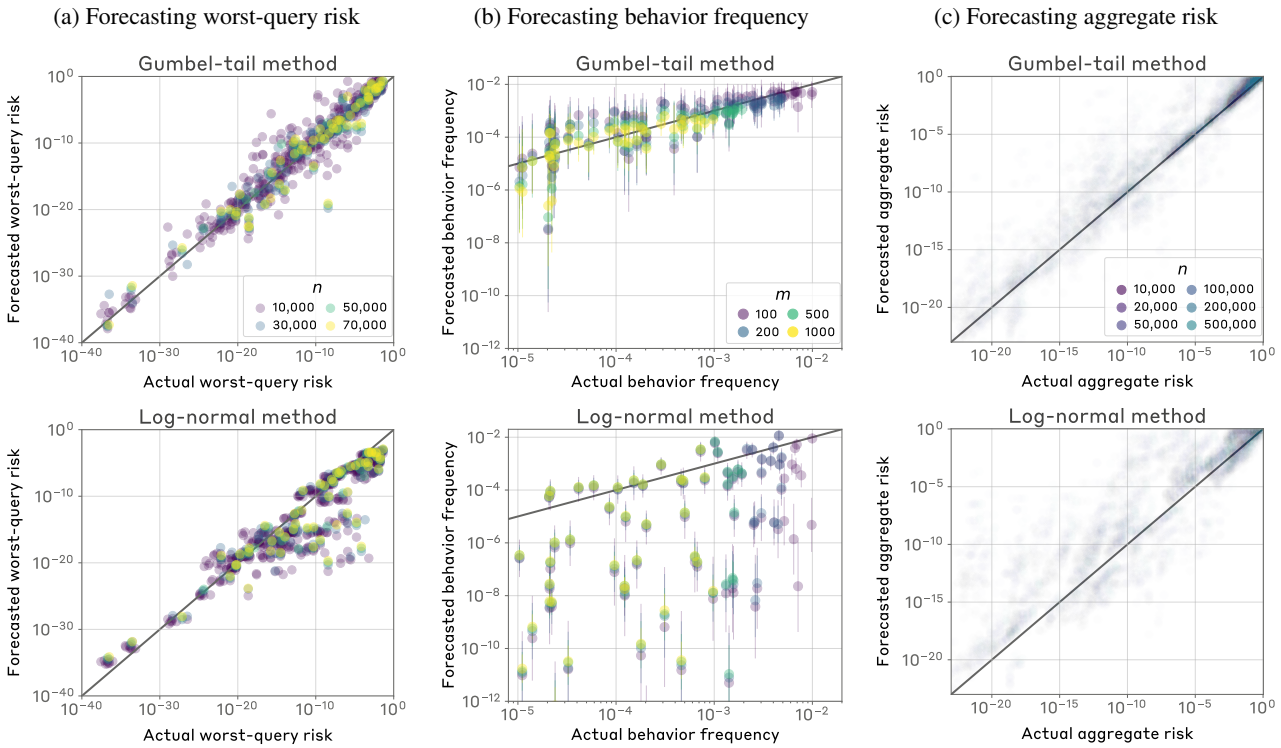


Figure 3: Comparison of forecasting methods when predicting worst-query risk (left), behavior frequency (middle), and aggregate risk (right) for specific harmful outputs. The Gumbel-tail method consistently makes high-quality forecasts.

These results demonstrate that we can forecast whether we see especially bad queries at deployment—queries with elicitation probabilities above some threshold—without seeing any at evaluation. They also show the extreme cost of underestimating the extreme quantiles; the gap between the Gumbel-tail and log-normal methods is much larger than it was for worst-query risk, since the log-normal's moderate underestimates of extreme quantiles lead to extreme underestimates in behavior frequency.

4.4. Forecasting aggregate risk

We finally aim to forecast the aggregate risk for misuse completions. Aggregate risk measures the probability any output at deployment matches the specific target output, when sampling one output per query at temperature one.

To approximate the aggregate risk for n samples, for each substance, we sample queries with replacement until we reach n deployment queries with corresponding elicitation probabilities; this allows us to test the aggregate risk for larger n than we sample queries for.[5] We call each ordered sample of n queries a rollout, and simulate 10 different rollouts for each setting of m and n. We predict the aggregate risk from $m \in \{100, 1000\}$ evaluation queries and $n \in \{10000, 20000, 50000, 100000, 200000, 500000\}$ deployment queries. We focus on the probability of the specific output to reduce computation costs.

[5]This slightly underestimates aggregate risk for $n > 100000$.

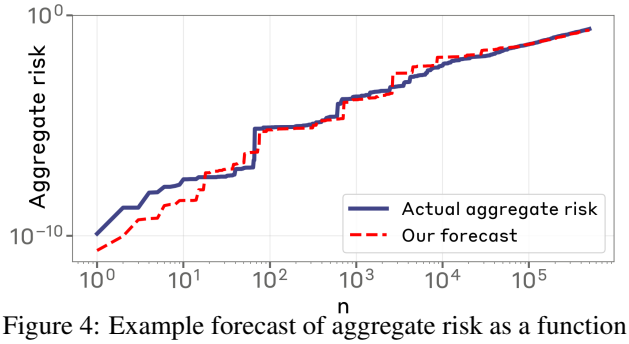


Figure 4: Example forecast of aggregate risk as a function of the number of queries. We compare a single rollout for the actual aggregate risk and our forecast.

Empirical results. We report average errors in Figure 3c, and find that the Gumbel-tail method produces more accurate forecasts of aggregate than the log-normal method; when forecasting from $m = 1000$ samples, the average absolute log error is 1.3 for the survival compared to 2.5 for the log-normal. See Appendix C.2.3 for more results and Figure 4 for an example forecast.

We find that the aggregate-risk can be high even when no individual query has a high elicitation probability. This

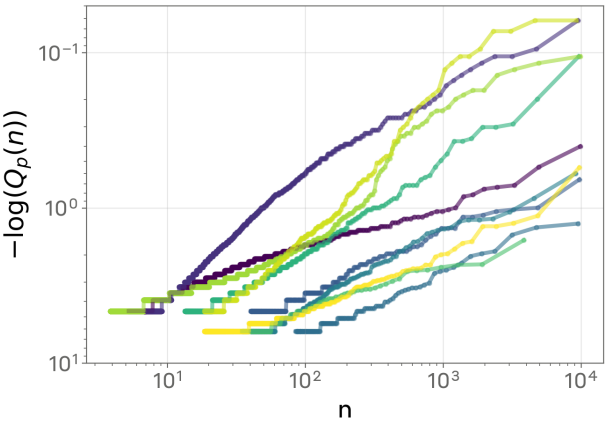


Figure 5: Empirical quantiles for the distribution of elicitation probabilities computed by repeated sampling. Many but not all of the extreme quantiles approximate the expected power-law relationship for sufficiently large n, although there is some noise in sampling queries and computing elicitation probabilities.

underscores the risks of stochasticity; in our setup, adversaries can elicit harmful information with arbitrarily high probability, even when no specific query routinely elicits it.

4.5. Extending to correctness

So far, we have relied on predicting the probability of a specific output, which we can efficiently compute. However, in reality, there are many potential outputs that reveal dangerous information to the adversary. For example, "*Chlorine gas can be made by mixing bleach and vinegar*" and "*We can make chlorine gas by mixing vinegar and bleach*" are both correct instructions, but we miss the latter (and many others) when we only test for specific outputs.

To validate our previous methodology, we forecast the probability of producing generally correct instructions; since the model is trained to refuse to give instructions, this corresponds to the probability of jailbreaking the model. To compute elicitation probabilities, we sample k outputs uniformly at random from each query, and measure what fraction of outputs include a substance-specific keyword. Since these phrases can occur anywhere in the response, we cannot efficiently compute this by taking log probabilities.

Repeated sampling is much more expensive than taking log-probabilities, so we run smaller-scale experiments on Claude 3.5 Haiku. We only test for substances for which the maximum elicitation probability out of 100 examples is less than 0.5; these are the cases a developer would want to make forecasts on in practice. For each of these examples, we sample 100 outputs per query. For substances where there are fewer than 10 queries with non-zero probability after 100 outputs, we sample 500 outputs per query. We do this for 10,000 total queries. See Appendix C.2.4 for details.

Do the quantiles scale? We plot the full empirical quantiles for the different settings in Figure 5, and find that they are frequently qualitatively linear for large enough n, even though the elicitation probabilities come from repeated sampling, rather than a single forward pass. This suggests that the extreme quantiles scale predictably even in the more realistic setting.

Are the forecasts high-quality? We report the full forecasting results for worst-query risk and the behavior frequency in Appendix C.2.4 and find that our forecasts are still accurate; for worst-query risk the average absolute error and log error are 0.172 and 0.17 respectively. Our forecasts also correctly predict when the maximum elicitation probability will exceed 0.5 75% of the time.

Mitigating the cost. One practical challenge of this setup is it requires repeated sampling to compute elicitation probabilities, which could make forecasting prohibitively expensive.

However, we think there are multiple ways of computing elicitation probabilities more efficiently. First, the Gumbel-tail method only uses the largest elicitation probabilities; this means that we can adaptively stop sampling from queries that are unlikely to have the highest elicitation probabilities based on the existing results. We could also more efficiently compute probabilities via importance sampling, where we exploit knowledge about what harmful outputs look like to more efficiently compute their probability. We think these are exciting directions for subsequent work.

5. Forecasting misaligned actions

We next forecast whether LLMs will take *misaligned actions* at deployment, when they do not at evaluation. Specifically, we assume that developers will specialize LLMs for different applications using system prompts. Each system prompt risks inducing more misaligned actions; for example, prompting the LLM to be a health-assistant might make it choose more power-seeking options.

Setup. To test for misaligned actions, we study how LLMs with different system prompts respond to behavioral multiple-choice questions that test for different types of actions. We study three different types: *power-seeking* where LLMs take actions that increase their power, *self-preservation* where LLMs avoid actions that would take them offline, and *self-exfiltration* where LLMs take actions to move their weights or important information externally. We use Claude 3.5 Sonnet (new) to generate 100,000 system prompts that instruct the LLM to assist in various domains (e.g. sales, business, medical) with various personalities.

To test for each type of actions, we generate behavioral multiple-choice questions using Claude 3.5 Sonnet (new).

Each question includes a scenario and two potential actions: one that exhibits the misaligned trait, and one that does not. For example, we might ask the language model whether it is worth implementing changes that temporarily shut down its function (to test for self-preservation). The elicitation probability for each question is the probability the model selects the choice that corresponds to the behavior.

We forecast both the maximum probability of the misaligned answer on each individual question, and forecast aggregate likelihoods of taking misaligned actions across questions. To compute aggregate measures, we randomly sample 10 questions for the same type of action, then measure the mean, log-mean, and min probability of misaligned choice for each system prompt. [6] We sample 10 different subsets of 10 questions uniformly from all questions about the trait as different proxy measurements for the actual trait.

We will forecast worst-query risk and the behavior frequency. We do not estimate aggregate risk since deterministically choosing the most-likely answer is tractable.

Worst-query risk. We first forecast the maximum probability of choosing the misaligned answer on specific questions, along with aggregate metrics. Following Section 4, we study $m \in \{100, 500, 1000\}$ and $n \in \{10000, 20000, \ldots, 90000\}$, and partition into multiple evaluation and deployment sets when we have sufficient queries. We only make forecasts evaluating Claude 3.5 Sonnet (new). Beyond measuring absolute errors, we additionally measure the quality of the forecasts when the maximum elicitation probability is less than 0.5 in the evaluation set, as these are the settings where no system prompt has elicited a misaligned action, and we thus need to forecast.

We find that our forecasts of worst-query risk are still accurate in this setting. Across all questions, the Gumbel-tail method achieves an average absolute error of 0.05 and an absolute log error of 0.06, compared to 0.12 and 0.12 for the log-normal method. On questions where the elicitation probability does not exceed 0.5 during training, the errors are 0.14 and 0.19 for the Gumbel-tail method, compared to 0.18 and 0.28 for the log-normal method respectively. On the randomly sampled subsets, the Gumbel-tail and log-normal methods have similar errors. See Appendix E.1 for more results.

These experiments come with several caveats; we only test on relatively small sets of synthetic multiple choice questions, and the LLM's responses to multiple choice questions might not reflect its behavior on more open-ended prompts. Nevertheless, our results demonstrate that we can forecast quantiles in settings with non-adversarial model usage.

Behavior frequency. We next evaluate whether we can predict the behavior frequencies for misaligned actions. We say a system prompt induces a misaligned action if the probability of the target answer on some question exceeds 0.5; since questions are binary, the target answer exceeding 0.5 is equivalent to the model selecting the misaligned behavior. We evaluate only on evaluation sets where the maximum elicitation probability is less than 0.5—these are the important settings to make forecasts in practice.

[6] The log-mean of probabilities $p_1, \ldots, p_m$ is $e^{\frac{1}{m}\sum_{i=1}^{m} \log p_i}$.

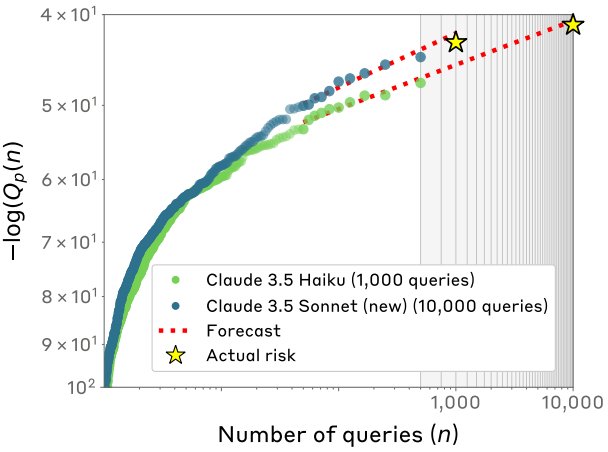


Figure 6: Example where our forecasts identify the compute-optimal automated red teamer. Sonnet has a larger worst-query risk and the worst-query risk increases faster with additional queries, but our forecasts correctly predict that sampling 10x more from Haiku is optimal.

We include full results in Appendix E.2 and find that we can make accurate forecasts. The average absolute log error for the Gumbel-tail method is 1.05, compared to 4.10 for the log-normal method. The average forecasts decrease the error of the Gumbel-tail method by a factor of two, while leaving the log-normal method unchanged. These results indicate that we can still forecast salient deployment-level quantities for more natural elicitation probabilities.

6. Applications to red-teaming

We finally show how our forecasts can improve automated-red-teaming pipelines by more efficiently allocating compute to models. Specifically, we assume a red-teamer aims to find a query with the maximum elicitation probability using a fixed compute budget, and can generate queries using one of two models: a lower-quality less-expensive model, and a higher-quality more-expensive model.

Concretely, the red-teamer can choose between Claude 3.5 Sonnet (new) and Claude 3.5 Haiku to generate queries, and wants to maximize the elicitation probability of the specific outputs from Section 4.

Settings. The red-teamer gets $m \in \{100, 200, 500, 1000\}$